Cụm 7 - Documenting Software Architecture

Mục lục

7.1 Tổng quan Cụm 7: Documenting Software Architecture

> **Tóm tắt một dòng**: Kiến trúc không được tài liệu hoá là kiến trúc chết - tồn tại trong đầu 1-2 người, evolve không kiểm soát, mất khi người đó rời team. Cụm này dạy framework chuẩn để document - 3 views (Module/C&C/Allocation), ADR, và diagram standards.

Vì sao quan trọng

3 lý do hard-hitting:

1. Bus factor

"Bus factor" = số người trong team rời đi (hypothetically bị bus đụng) để hệ thống không ai biết. Bus factor = 1 nghĩa hệ thống chỉ 1 người hiểu. Đó là disaster waiting to happen.

Document tốt □ bus factor ≥ 3. Team thay đổi, kiến trúc không bị lost.

2. Onboarding

New engineer join team. Không document □ spend 2-3 tháng "figure out" hệ thống qua đọc code. Có document tốt □ onboard 2 tuần.

Cost difference: ~3 tháng * salary mỗi engineer.

3. Architecture evolution

Quyết định cũ làm vì context cũ. Context thay đổi (vd: business model pivot) □ cần revisit decision. Không document *vì sao* □ không biết quyết định nào còn valid.

ADR (Architecture Decision Record) chính là giải pháp.

Cụm 7 sẽ dạy

Bốn bài:

7.2: Module Views

View về *static structure* của code. Module/package/class hierarchy. Subtypes: decomposition, dependency, generalization, uses. C4 model (Context-Container-Component-Code).

7.3: Component-and-Connector Views

View về *runtime* structure. Component instances, connectors, data flow. Subtypes: process, concurrency, communication, deployment-time.

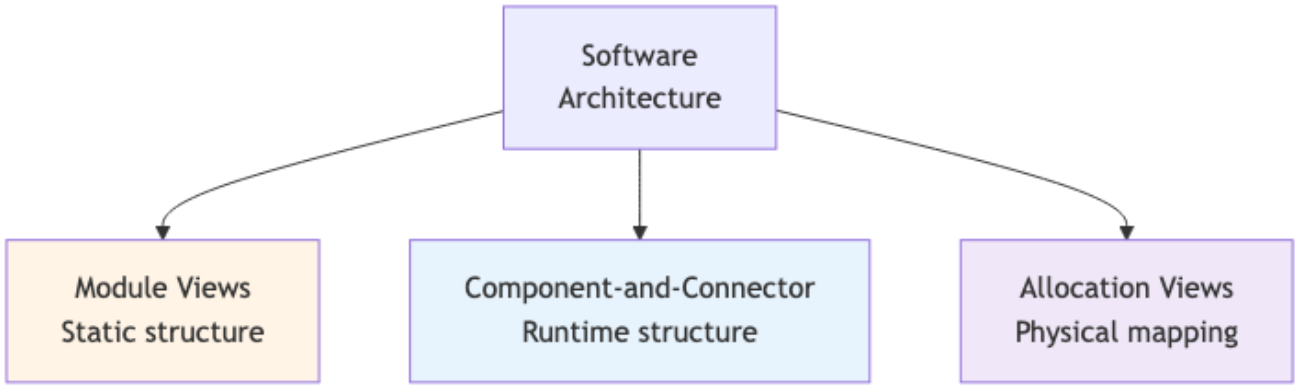
7.4: Allocation Views

View về *physical deployment*. Mapping software □ hardware/network/people. Subtypes: deployment, implementation, work assignment.

Plus: ADR (Architecture Decision Records) lồng vào Bài 7.2.

Framework chính: 3 views (Bass-Clements-Kazman)

SEI Carnegie Mellon trong *Software Architecture in Practice* định nghĩa 3 view fundamental:



Hình 1: Sơ đồ 27

Mỗi view answer câu hỏi khác:

- **Module**: Code organized ra sao? Module nào depend module nào?
- **C&C**: Runtime hệ chạy ra sao? Component nào communicate với component nào, khi nào?
- **Allocation**: Software chạy ở đâu? Mapping vào server/cloud/team?

Documenting đầy đủ ≥ 1 view mỗi loại. Big system có thể có nhiều view mỗi loại.

Diagram standards

Đừng vẽ diagram ad-hoc. Dùng standard để team đọc được:

C4 Model (Simon Brown)

4 mức zoom:

1. **System Context**: hệ trong landscape (user, external system).
2. **Container**: app, database, broker. Mỗi container = deployable unit.

3. **Component**: bên trong container, các component logic.
4. **Code**: class, function (rarely drawn, code is source of truth).

C4 đơn giản, phổ biến, free tool (c4-plantuml, structurizr).

4+1 View (Philippe Kruchten, 1995)

5 view:

- **Logical**: object-oriented design (UML class diagram).
- **Process**: runtime processes (UML sequence/activity).
- **Development**: code organization (UML package/component).
- **Physical**: deployment (UML deployment diagram).
- **Scenarios**: use cases tying everything together.

4+1 chi tiết hơn C4, more formal. Phù hợp enterprise documentation.

UML

Universal Modeling Language. Set of diagram types. Vẫn dùng nhưng heavy. Modern team thường dùng C4 instead.

Mermaid / PlantUML

Tools render diagram from text. Maintainable in git, version-controlled.

```
graph TD
    A[Web App] --> B[API Gateway]
    B --> C[Order Service]
```

Phù hợp markdown documentation.

ADR, Architecture Decision Record

Đã giới thiệu Bài 3.3. Format:

```
# ADR-N: Title

## Status
Proposed | Accepted | Deprecated | Superseded

## Context
What's the situation? Why do we need to decide?

## Decision
What did we decide?

## Consequences
Positive + negative consequences.

## Alternatives Considered
What else we considered, why rejected.
```

ADR file numbered, append-only (don't edit accepted ADRs; superseded by new ADR).

Vì sao ADR works:

- **Lightweight**: < 1 trang. Engineer chịu viết.
- **In git**: version-controlled với code. Diff over time.
- **Searchable**: future engineer find why decision was made.

Tools: adr-tools (CLI), Markdown ADR convention.

Practices documentation

Living documentation

Doc evolve cùng code. CI verify (vd: link checker, snippet test).

Audience-aware

Different doc cho different audience:

- **Architect / Lead**: full diagram + ADR.
- **Engineer**: code-level doc, API spec.
- **Onboarding**: tutorial, walk-through.
- **Stakeholder / PM**: high-level diagram + narrative.

Minimum viable doc

Đừng over-document. Default:

- 1 System Context diagram.
- 1 Container diagram (C4).
- Module/C&C view cho khu phức tạp.
- ADRs cho important decisions.
- README per repo: setup, build, contribute.

Không cần full UML mọi class.

Cách đọc cụm

Tuần tự 7.2 □ 7.3 □ 7.4. Mỗi bài ~3000-4000 chữ.

Sau cụm, bạn nên:

- Vẽ được Module View + C&C View + Deployment View cho hệ mình đang làm.
- Viết được 5 ADR cho 5 quyết định gần nhất.
- Setup C4 diagram trong markdown của repo.

Bài tiếp: Module Views.

7.2 Module Views

> **Tóm tắt một dòng**: Module View show static structure của code - module nào tồn tại, depend module nào. Quan trọng cho developer hiểu codebase và cho architect verify modularity (Cụm 3.4).

Module là gì trong context view này

Module = implementation unit. Có thể là:

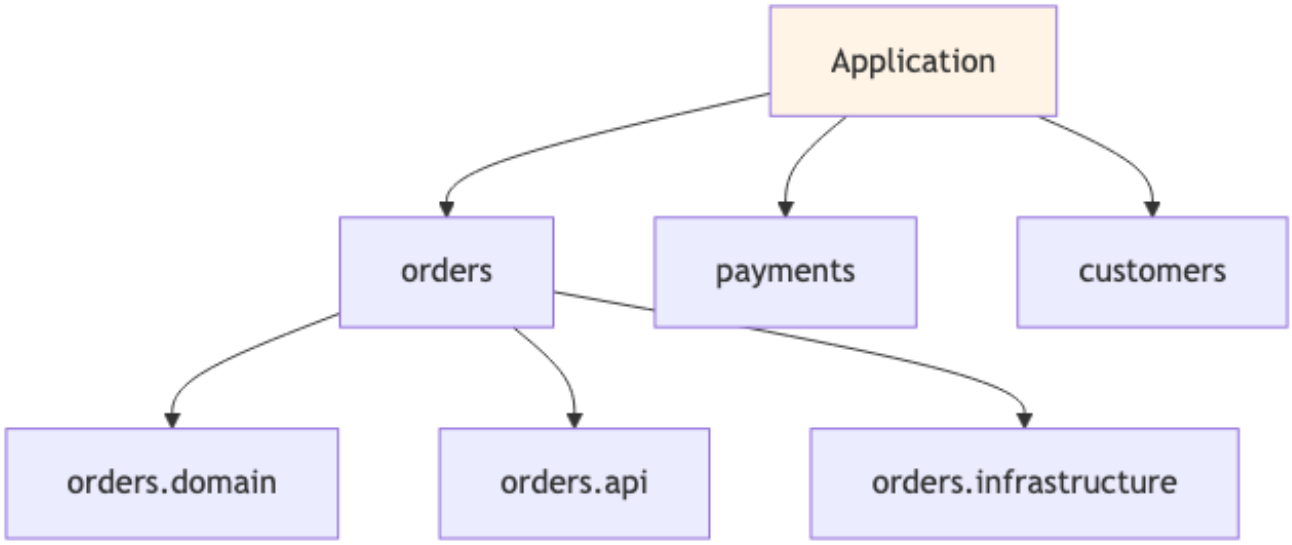
- Package / namespace (Java package, Python module, C# namespace).
- Library / module (npm package, Maven module).
- Class (lớn).
- Subsystem.

Scale: bài này focus mức package/library, không quá fine-grained (không vẽ mọi class).

4 subtypes của Module View

Subtype 1: Decomposition View

Hiển thị hierarchy "module này chứa các sub-module nào".



Hình 2: Sơ đồ 28

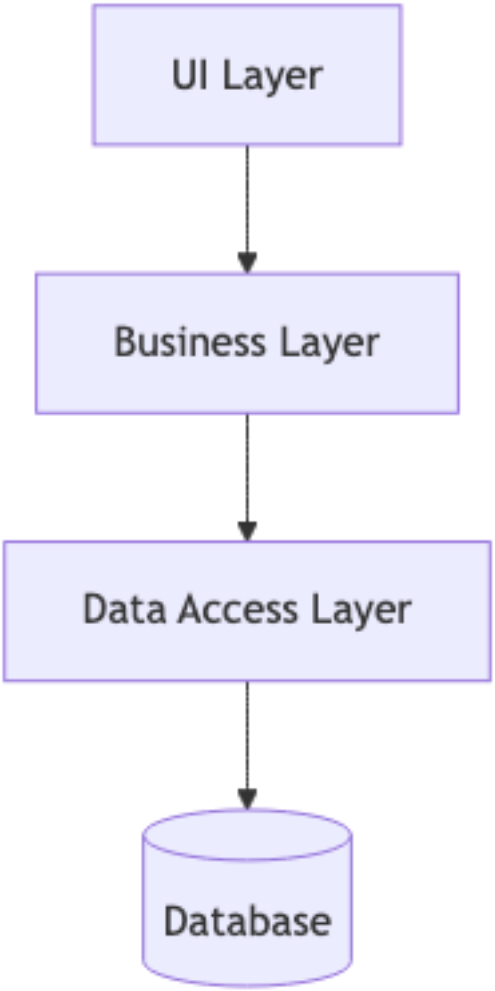
Purpose: navigation. Developer mới hiểu "code organize ở đâu".

Subtype 2: Dependency View

Show "module nào depend module nào".

Purpose: verify modularity. Architect check: có cycle không? Direction có đúng không?

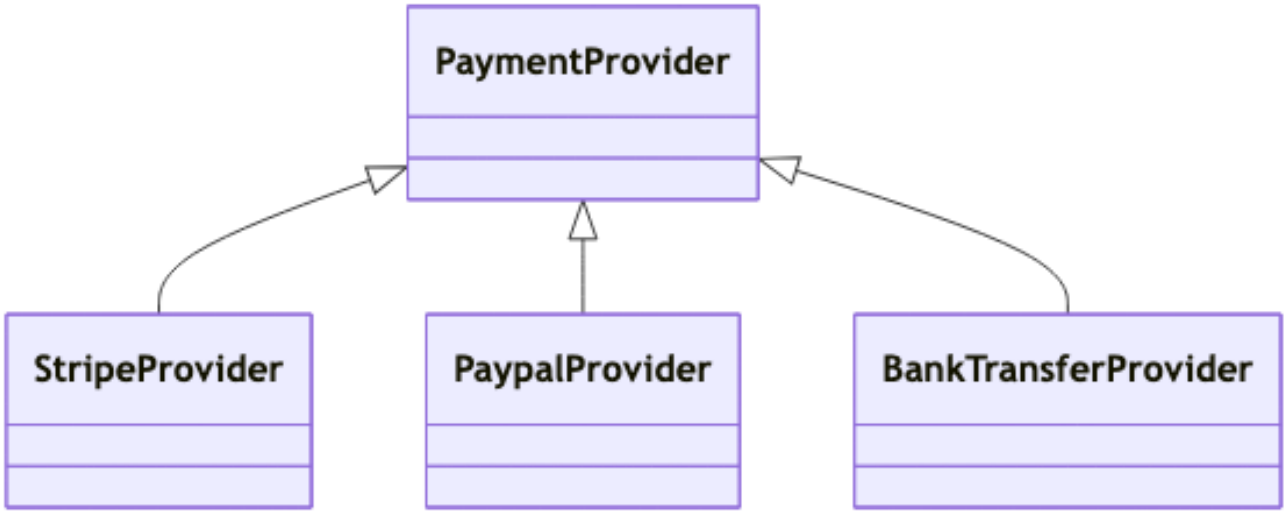
Tools: `madge` (JS), `pydeps` (Python), `jdeps` (Java) generate auto.



Hình 3: Sơ đồ 29

Subtype 3: Generalization View

Show inheritance / interface hierarchy.



Hình 4: Sơ đồ 30

Purpose: design review. Verify OCP/LSP applied correctly.

Subtype 4: Uses View

Show “module này uses module nào” (richer than dependency). Ai gọi ai ở mức operation.

Purpose: impact analysis. “Nếu sửa module X, module nào có thể bị ảnh hưởng?”.

C4 Model deep dive

C4 (Simon Brown) là chuẩn modern phổ biến nhất. 4 mức:

Level 1: System Context

Show hệ ở giữa, surrounded by users + external systems.

Sơ đồ Mermaid (không render được, xem trên web)

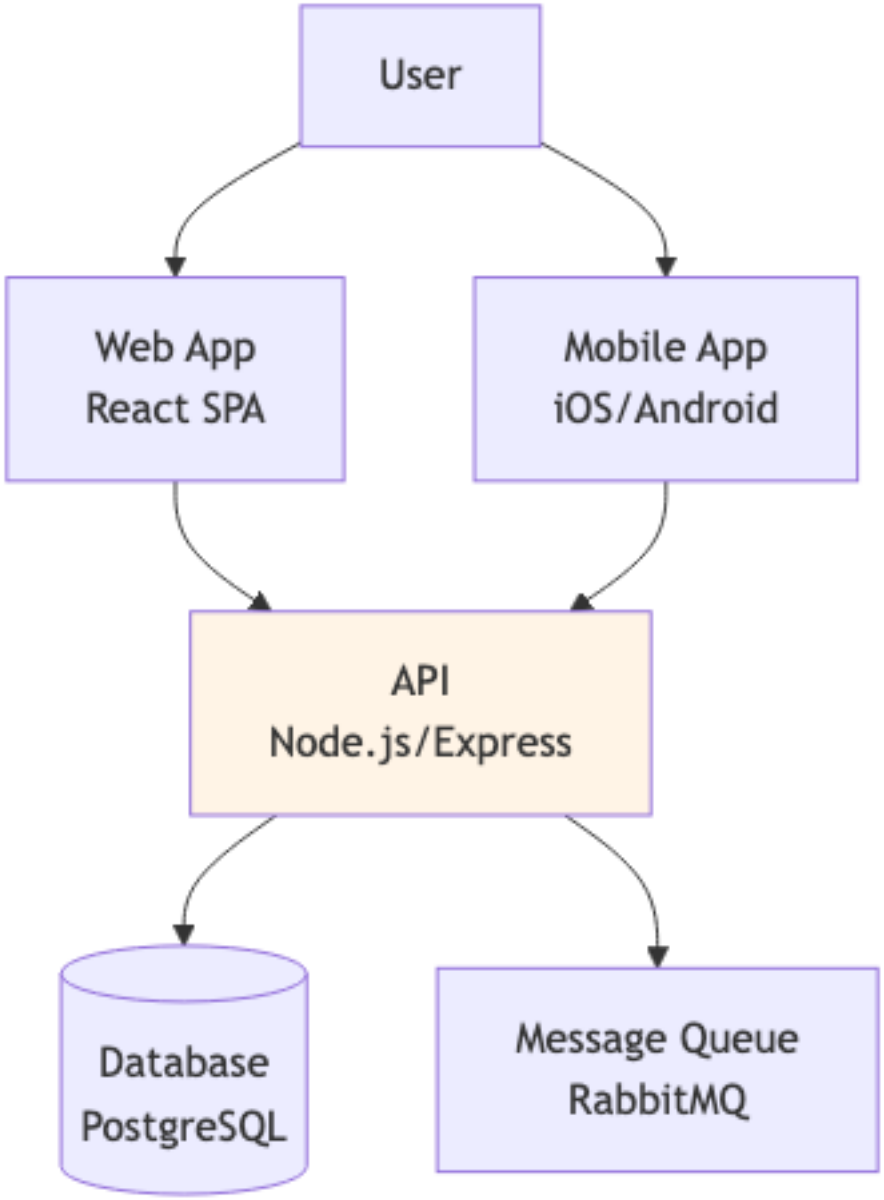
```
graph TD
    U[User]
    S[Our E-commerce<br/>System]
    E1[Stripe<br/>Payment]
    E2[Inventory<br/>System (legacy)]
    E3[Email Service]

    U -->|browses, orders| S
    S -->|charge| E1
    S -->|check stock| E2
    S -->|send confirmation| E3
```

```
    style S fill:#fff4e6
```

Audience: stakeholder, PM. Mục đích: “bigger picture”.

Level 2: Container

Zoom vào System box □ show containers. Container = deployable unit (web app, mobile app, database, message broker).

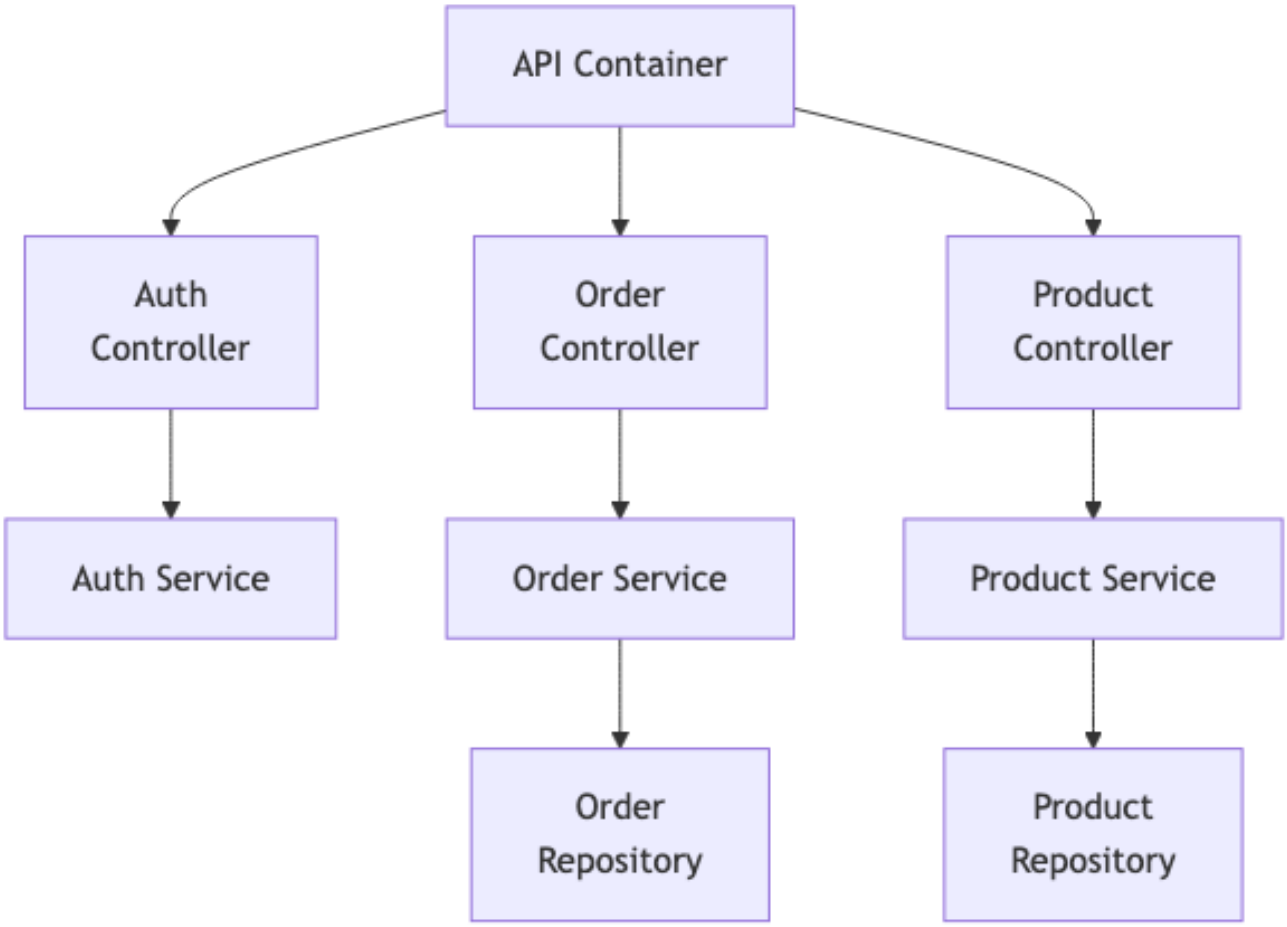


Hình 5: Sơ đồ 32

Audience: architect, dev lead. Mục đích: hiểu tech stack + deployment.

Level 3: Component

Zoom vào 1 container □ show components bên trong.



Hình 6: Sơ đồ 33

Audience: developer. Mục đích: hiểu code structure trong container.

Level 4: Code (optional)

UML class diagram. Hiếm khi vẽ, code là source of truth.

Tools cho C4

- **Structurizr DSL** (Simon Brown): text-based, render mọi level.
- **PlantUML + c4-plantuml**: include macro để vẽ C4.
- **Mermaid**: như example trên, dễ render trong markdown.
- **Draw.io**: GUI, nhưng không version-control friendly.

Recommendation: Structurizr DSL hoặc Mermaid + PlantUML.

ADR, Architecture Decision Record

ADR doc dạng:

```
# ADR-007: Use PostgreSQL for primary data store

## Status
Accepted (2026-01-15)

## Context

We need to select a primary database for the e-commerce platform. Expected scale:
- 50k users in year 1, target 500k by year 3.
- Complex queries (joins for product search, analytics).
- ACID required for orders and payments.
- Team experience: strong in SQL, basic in NoSQL.

## Decision

Use PostgreSQL 16 as primary data store. Use Redis for caching layer.

## Alternatives Considered

### MongoDB
- Pros: easier schema evolution, native JSON.
- Cons: weak ACID for cross-doc transactions, team less experienced.
- Rejected: ACID critical for payments; team expertise.

### MySQL
- Pros: similar SQL, mature.
- Cons: weaker JSON support, less feature-rich than Postgres.
- Rejected: Postgres has better JSONB, better extensions.

### DynamoDB
- Pros: managed, scalable.
- Cons: complex query model, vendor lock.
- Rejected: requires schema redesign around access patterns.

## Consequences

### Positive
- Strong ACID for orders/payments.
- Rich query support for analytics.
- Team productive immediately.

### Negative
- Scaling beyond ~1M users may require sharding (vs DynamoDB auto-scale).
- Operations overhead (backup, replication, tuning).

## Mitigation
- Use managed Postgres (AWS RDS or Aurora) to reduce ops burden.
- Plan for read replica at 100k users.
```

```
- Plan for sharding strategy at 500k users.
```

ADR best practices

1. **Number sequentially**: ADR-001, ADR-002, … never reuse numbers.
2. **Append-only**: Don't edit accepted ADRs. Supersede with new ADR.
3. **Short**: ≤ 1 page. Engineers won't read longer.
4. **Decision-focused**: 1 ADR = 1 decision, not analysis paper.
5. **Linked from code**: vd: PR mention ADR-007.
6. **Searchable**: keep in repo `docs/adr/`, in markdown.

Tools

- **adr-tools** (CLI): `adr new "Use PostgreSQL"`.
- **GitHub markdown**: render natively.
- **MADR template** (markdown ADR): widely-used template.

Practical: Documenting một service

Mức minimum cho 1 microservice:

1. **README.md**: setup, build, contribute.
2. **docs/architecture.md**: 1 page với:
   - System Context (where this service fits).
   - Container (deployment).
   - Key data models (top 5 entities).
   - Key flows (top 3-5 sequence).
3. **docs/adr/**: ADR file cho important decisions.
4. **API spec**: OpenAPI/protobuf in repo.
5. **runbook.md** (cho on-call): common ops procedures.

Tổng < 2 hours setup, value cho team rất cao.

Anti-patterns

Anti-pattern 1: Big upfront design

Write 200-page architecture document, then build. By time build done, doc outdated. No one reads.

Fix: living doc. Update with code changes. Use ADR for decisions.

Anti-pattern 2: Diagram-only doc

Doc chỉ diagram, không có narrative. Audience không hiểu context.

Fix: combine diagram + paragraph. "Đây là hệ X, làm Y, ở high level chia thành Z. [diagram]. Chi tiết: …".

Anti-pattern 3: No audience awareness

Same doc cho mọi audience. Quá technical cho stakeholder, quá high-level cho dev.

Fix: layered doc. Stakeholder doc (C4 Context), Architect doc (C4 Container + ADR), Dev doc (C4 Component + code).

Anti-pattern 4: Doc rot

Doc viết 1 lần rồi không update. Sau 1 năm, doc ≠ code. Worse than no doc.

Fix: doc trong git. Code review verify doc updated. CI link checker.

Tóm tắt

- **Module Views**: static code structure. 4 subtypes.
- **C4 Model**: 4 mức zoom (Context-Container-Component-Code).
- **ADR**: 1-page decision record, append-only, in git.
- **Practical minimum**: README + architecture.md + ADR + API spec + runbook.
- **Tránh**: big upfront, diagram-only, no audience awareness, doc rot.

Bài tiếp: C&C Views, runtime structure.

7.3 Component-and-Connector Views

> **Tóm tắt một dòng**: C&C View show *runtime* structure - khi hệ chạy, component instances nào tồn tại, communicate với connector nào, data flow ra sao. Khác Module View ở chỗ Module = static, C&C = dynamic.

Module vs C&C

Aspect	Module View	C&C View
Element	Module (class, package)	Component instance, connector
Relation	Depend, contain, inherit	Communicate at runtime
Time	Static (code)	Dynamic (runtime)
Example	Order package depends Payment package	OrderService instance calls PaymentService via HTTP
Tool	C4 Component diagram, UML class	Sequence diagram, runtime topology

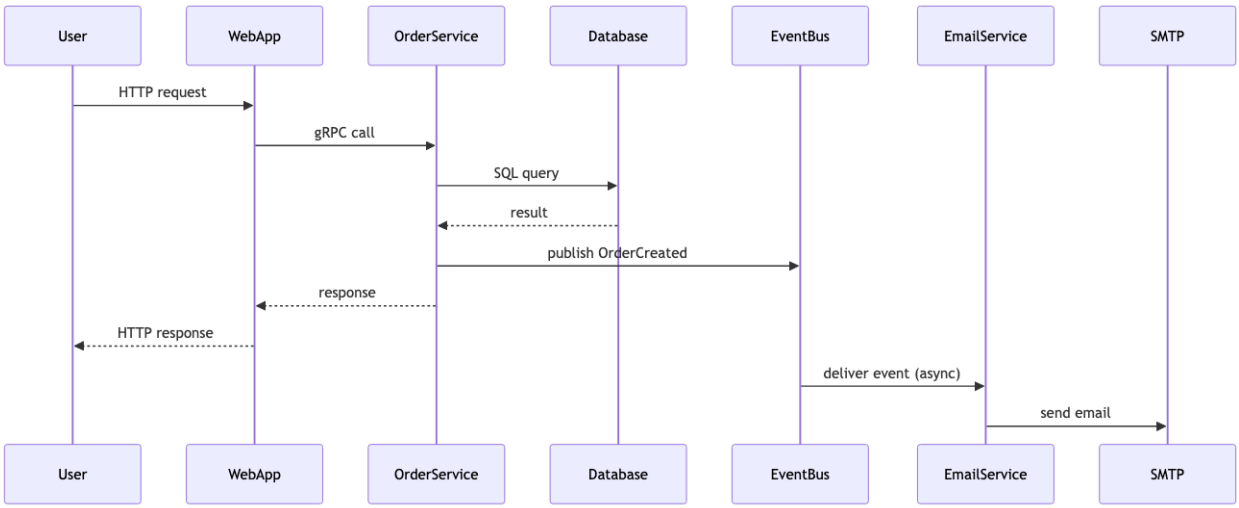
Cùng hệ có thể có Module và C&C view *khác nhau*. Vd:

- Module: 1 `payment` module exist.
- C&C: 3 instance PaymentService chạy parallel (replicated for HA).

Subtypes của C&C

Subtype 1: Process View

Show processes/threads chạy ở runtime + sync/communication.



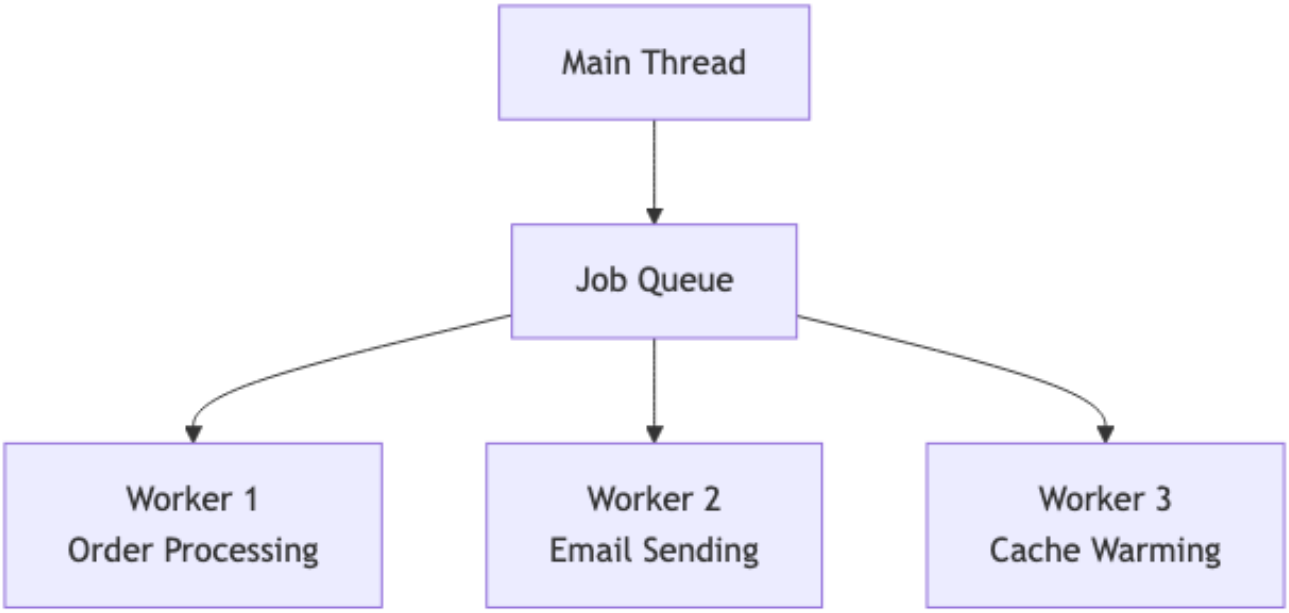
Hình 7: Sơ đồ 34

Purpose: hiểu request flow, latency contributors.

Subtype 2: Concurrency View

Show concurrent execution + synchronization.

Vd:

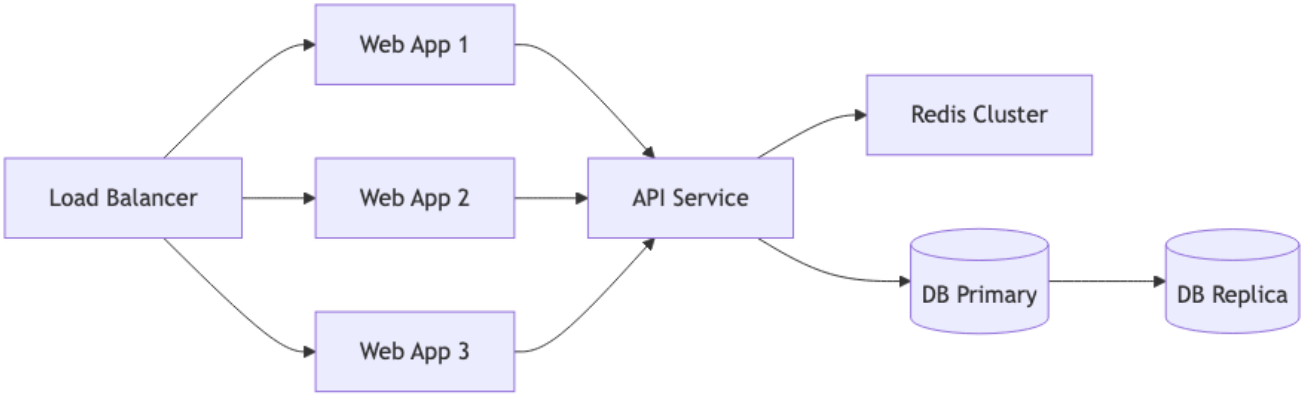


Hình 8: Sơ đồ 35

Purpose: identify race condition, deadlock risk, sync points.

Subtype 3: Communication View

Show topology của component instances + communication links.



Hình 9: Sơ đồ 36

Purpose: visualize HA, scalability, single points of failure.

Subtype 4: Deployment-time

Same as Allocation View (Bài 7.4). Where component instances run physically.

Khi nào dùng C&C View

- **Debug performance**: trace request flow.
- **Identify bottleneck**: where data flows slow.
- **Design HA**: show replication, failover paths.
- **Onboard SRE**: where data flows through stack.
- **Capacity planning**: scale point identification.

Diagram types

Sequence Diagram (UML)

Time vs participant. Show message exchange.

Pros: rất intuitive cho dynamic behavior.

Cons: only 1 scenario per diagram. Many scenarios = many diagrams.

Tools: Mermaid, PlantUML, draw.io.

Activity Diagram (UML)

Flow chart cho process. Decision, loop, parallel.

Pros: tốt cho business workflow.

Cons: less precise for technical detail.

Communication Diagram (UML)

Component + numbered messages.

Pros: emphasize structure.

Cons: less popular than sequence.

Runtime Topology Diagram

Boxes + connectors, custom notation.

Pros: flexible.

Cons: không standard, learning curve cho reader.

C4 Container diagram cũng là dạng C&C View ở mức container.

Connector types

Connector = mechanism component giao tiếp:

- **Method call** (in-process).
- **HTTP/REST**: sync, request/response.
- **gRPC**: sync, binary protocol.
- **Message queue**: async (Kafka, RabbitMQ).
- **Event bus**: async pub/sub (EDA).
- **Database call**: query.
- **File system**: read/write file.

- **Shared memory**: in-process or IPC.

C&C diagram nên *label* connector với type:

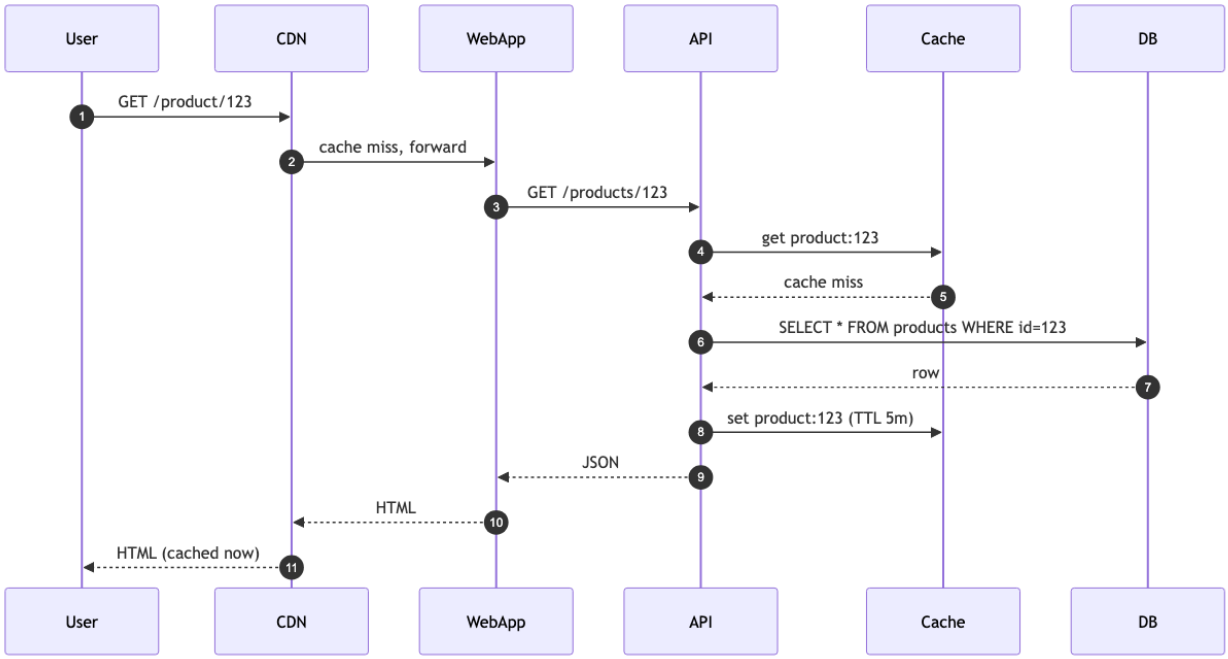
```
[OrderService] --HTTP--> [PaymentService]
[OrderService] --Kafka--> [EmailService]
[WebApp] --gRPC--> [API]
```

Label giúp reader hiểu communication semantics (sync vs async, latency tier).

Documenting common runtime patterns

Pattern 1: Request flow

Trace 1 user request qua stack:



Hình 10: Sơ đồ 37

Use case: explain to new dev, debug latency issue.

Pattern 2: Failover

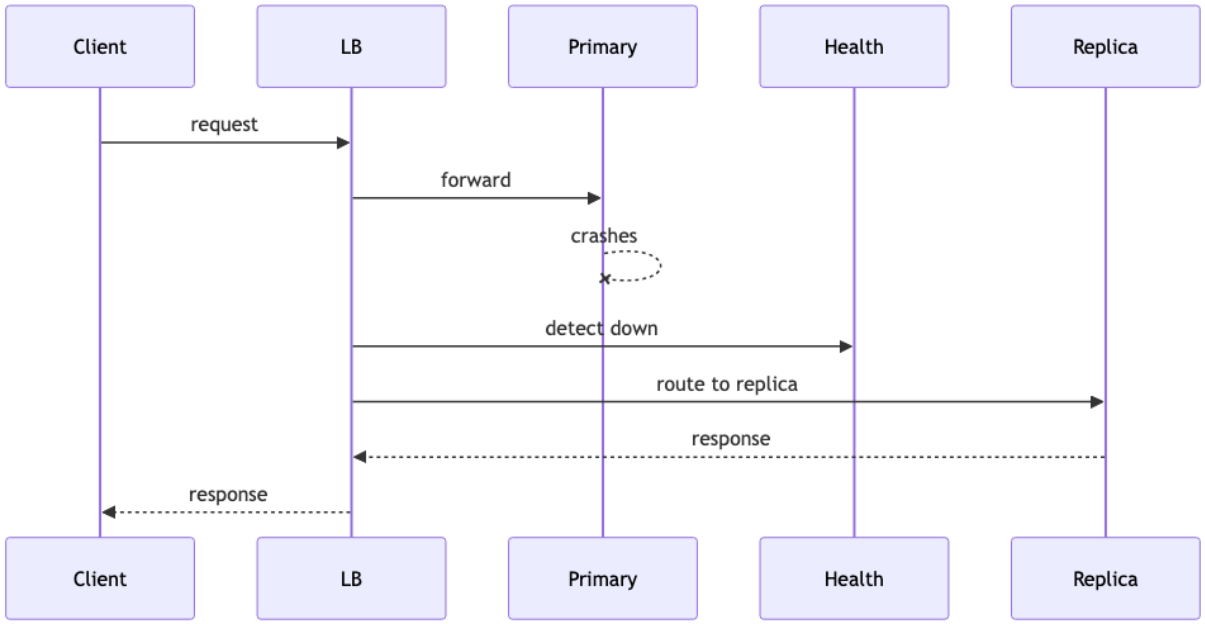
Use case: explain HA strategy to architect review.

Pattern 3: Saga (distributed transaction)

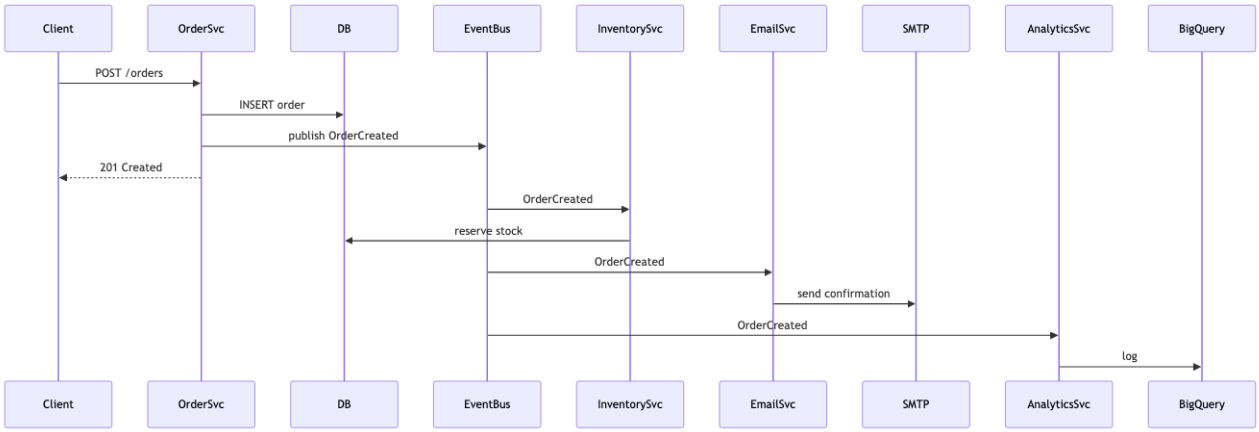
Đã show ở Bài 6.4.

Pattern 4: Async event flow

Use case: explain decoupling of producer from consumers.



Hình 11: Sơ đồ 38



Hình 12: Sơ đồ 39

Sai lầm thường gặp

Sai lầm 1: Vẽ mọi sequence

100 sequence diagrams cho 100 endpoints. Maintain nightmare.

Fix: chỉ vẽ top 5-10 critical/complex flows. Đơn giản flows = doc trong API spec.

Sai lầm 2: Quên label connector

“A □ B” không biết HTTP hay async event. Reader misunderstand.

Fix: always label arrow type.

Sai lầm 3: Mixed level of abstraction

Một diagram show high-level (services) lẫn low-level (function call). Confusing.

Fix: each diagram one level. C4 model là nguyên tắc tốt.

Sai lầm 4: No legend

Diagram phức tạp với colors/shapes mà không có legend. Reader guess meaning.

Fix: legend mọi non-standard notation.

Sai lầm 5: Outdated runtime

Code thay đổi, diagram không update. Worse than no diagram.

Fix: include diagram update in PR template for major changes.

Tóm tắt

- **C&C View**: runtime structure (vs Module = static).
- **4 subtypes**: Process, Concurrency, Communication, Deployment.
- **Diagram types**: sequence (most popular), communication, runtime topology.
- **Always label connectors** với type (HTTP/Kafka/gRPC/…).
- **Tránh**: vẽ mọi sequence, mixed abstraction, outdated.

Bài tiếp: Allocation Views, physical deployment.

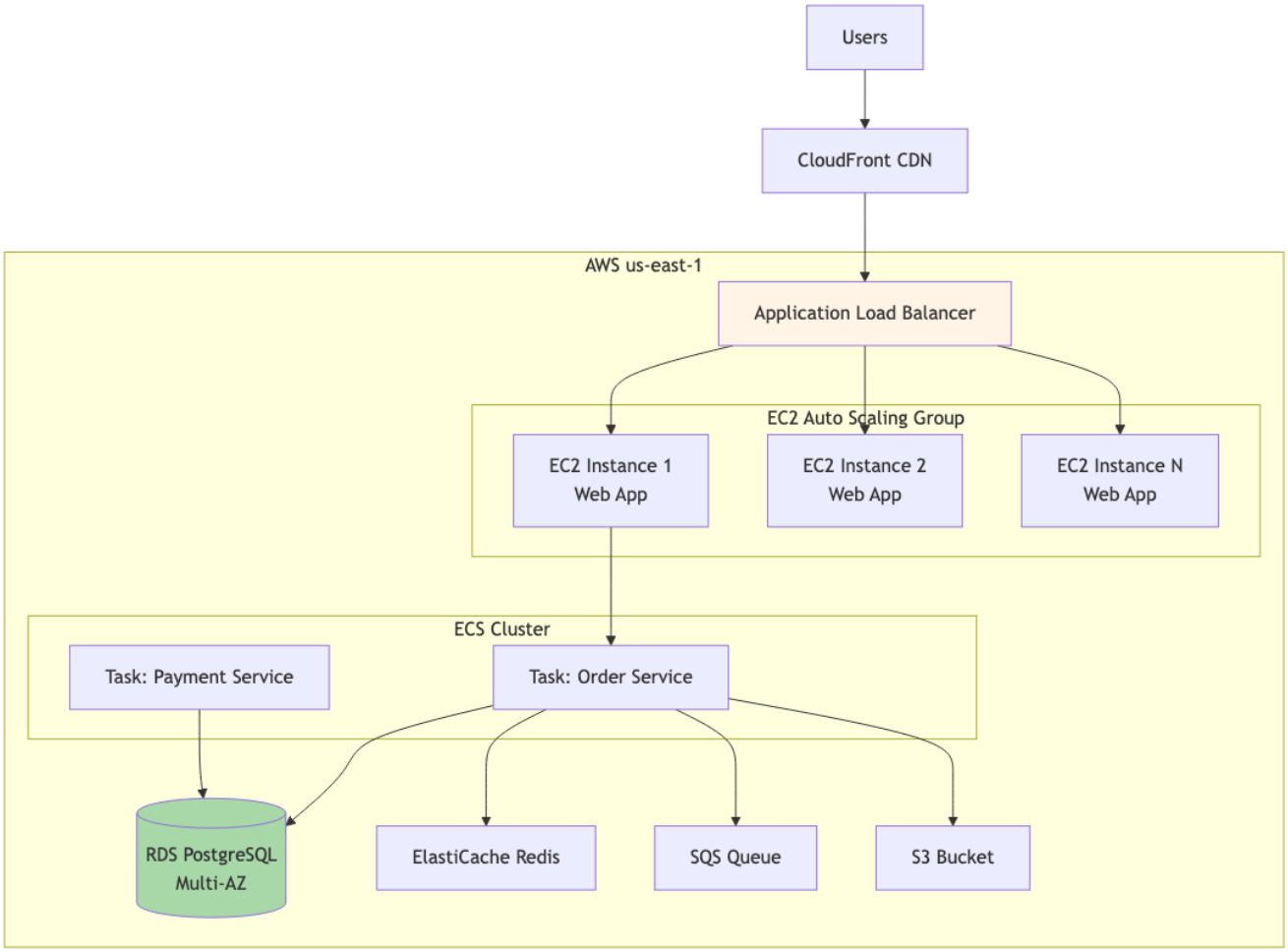
7.4 Allocation Views

> **Tóm tắt một dòng**: Allocation View show *physical* mapping của software lên: hardware/cloud (deployment), file system (implementation), team (work assignment). Trả lời “code này chạy ở đâu, ai own nó”.

3 subtypes

Subtype 1: Deployment View

Mapping software □ hardware/cloud infrastructure.



Hình 13: Sơ đồ 40

Audience: SRE, DevOps, architect.

Purpose:

- Capacity planning.
- Cost estimation.
- Failure scenario analysis.
- Migration planning.

Subtype 2: Implementation View

Mapping software □ file system / repository structure.

```
e-commerce-platform/
├── apps/
|   ├── web/                    # React SPA
|   ├── mobile/                 # React Native
|   └── admin/                  # Admin UI
├── services/
|   ├── order-service/          # Go service
|   ├── payment-service/        # Java service
|   ├── inventory-service/      # Python service
|   └── notification-service/   # Node.js service
├── shared/
|   ├── proto/                  # gRPC schemas
|   ├── events/                 # Event schemas
|   └── libs/                   # Shared libraries
├── infra/
|   ├── terraform/              # Infrastructure as Code
|   ├── helm/                   # K8s charts
|   └── monitoring/             # Grafana, Prometheus config
├── docs/
|   ├── architecture/           # ADRs, diagrams
|   └── runbooks/               # Ops runbooks
└── README.md
```

Audience: developer, new joiner.

Purpose:

- Navigation: “find code”.
- Mono vs multi repo decision documentation.

Subtype 3: Work Assignment View

Mapping software □ team ownership.

Team	Owns
Order team (5 engineers)	order-service, order-events, order admin UI
Payment team (4 engineers)	payment-service, payment integrations
Inventory team (3 engineers)	inventory-service, warehouse interface
Platform team (6 engineers)	shared libs, monitoring, CI/CD
Web team (4 engineers)	web SPA, mobile, design system

Audience: Engineering Manager, Director, new joiner.

Purpose:

- On-call rotation.

- Code review routing.
- Hiring plan.
- Conway's law analysis.

Cloud topology patterns

Common patterns trong deployment view:

Pattern 1: Single region, multi-AZ

Hệ chạy ở 1 region (us-east-1), span 3 availability zones cho HA.

Use case: hầu hết app medium scale, không cần multi-region.

Cost: medium. Failover: AZ-level (region down = total down).

Pattern 2: Multi-region active-passive

Primary region serves traffic. Secondary region standby, can take over within minutes.

Use case: critical app cần disaster recovery.

Cost: high (secondary region underutilized).

Pattern 3: Multi-region active-active

Both regions serve traffic. Replication bidirectional.

Use case: global app, low latency cho global users.

Cost: highest. Complexity: high (data sync).

Pattern 4: Multi-cloud

Spread across AWS, GCP, Azure.

Use case: avoid vendor lock-in, regulatory.

Cost: very high. Operational complexity: very high.

Pattern 5: Edge + Cloud

Static assets ở CDN edge (CloudFront, Fastly). Dynamic logic ở core cloud.

Use case: any web app. Standard practice.

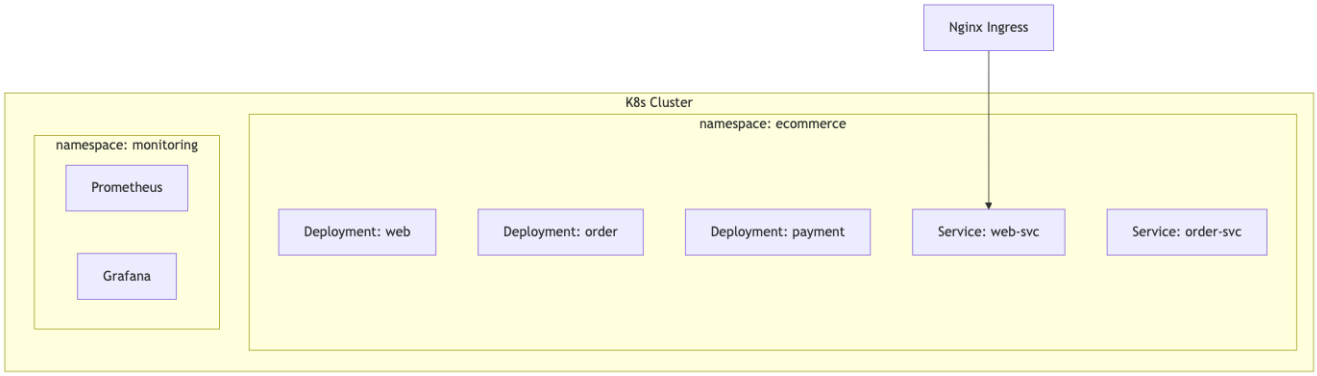
Container orchestration

Modern deployment thường dùng container orchestration:

Kubernetes (K8s)

Standard cho large-scale container deployment. Show as:

Audience: SRE, K8s admin.



Hình 14: Sơ đồ 41

Managed services

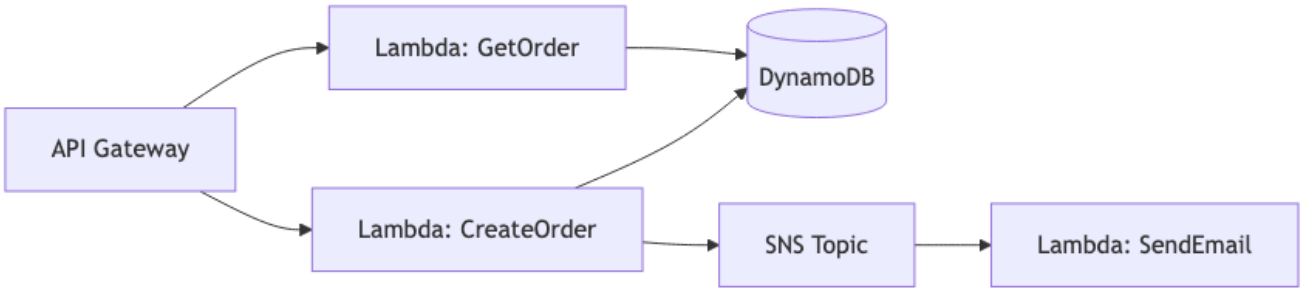
Cloud-native: AWS App Runner, GCP Cloud Run, Fly.io. Simpler ops, less control.

Use khi: team không có K8s expertise.

Serverless

AWS Lambda, GCP Cloud Functions. No server management.

Show as:



Hình 15: Sơ đồ 42

Use case: event-driven, low-medium traffic, cost-sensitive.

Conway's Law

Melvin Conway (1968):

> "Organizations which design systems are constrained to produce designs which are copies of the communication structures of these organizations."

Diễn đạt: architecture của hệ phản ánh org chart của team build.

Examples

- 3-team org (frontend, backend, mobile) □ 3-tier architecture với boundary tại tech layer.
- Bounded context per team □ microservices boundary follow team.

- Centralized DBA team □ shared database (DBA gatekeeper).

Implications

- **Architecture decision = org decision**. Đổi từ monolith □ microservices ≈ đổi org từ functional □ domain team.
- **Inverse Conway maneuver**: tổ chức org theo mong muốn architecture, để Conway force tự nhiên tạo ra architecture đó.

Visualization

Work Assignment View chính là Conway's law trong tài liệu. Show team-component mapping □ architect predict architecture evolution.

Documenting cloud cost

Allocation View nên include cost (estimate):

Component	Instance	Cost/month
Web App (3x EC2 t3.medium)	EC2	$90
Order Service (ECS Fargate, 2 task)	ECS	$120
Database (RDS PostgreSQL, m5.large)	RDS	$250
Redis (ElastiCache cache.t3.small)	ElastiCache	$25
Load Balancer	ALB	$20
Data transfer (1TB out)	Network	$90
Total		**~$595/month**

Cost transparency helps:

- Architect cân nhắc cost trong decision (Bài 3.3 trade-off).
- Engineering Manager budget planning.
- Identify waste (under-utilized resources).

Tools: AWS Cost Explorer, Cloud Custodian, custom dashboard.

Sai lầm thường gặp

Sai lầm 1: Outdated deployment diagram

Infra thay đổi (added Redis, swapped DB), diagram không update.

Fix: infra-as-code (Terraform) is source of truth. Diagram auto-generate from IaC nếu possible.

Sai lầm 2: No cost annotation

Diagram show topology mà không cost. Decision purely technical, ignore $$.

Fix: annotate cost per component (rough estimate OK).

Sai lầm 3: Quên failure scenario

Diagram show happy path. Không show “DB down thì sao”, “AZ down thì sao”.

Fix: separate diagram cho failure scenarios. Annotate SLA.

Sai lầm 4: Work Assignment không up-to-date

Org chart thay đổi (team merge, split), document outdated.

Fix: review work assignment view quarterly. Update on org change.

Sai lầm 5: Quên Conway’s law

Đổi architecture mà không đổi team. Distributed monolith result.

Fix: align team boundary với architecture boundary. Inverse Conway.

Tóm tắt

- **Allocation Views**: software □ physical/team mapping.
- **3 subtypes**: Deployment (hardware), Implementation (file system), Work Assignment (team).
- **Cloud patterns**: single/multi region, active-active, multi-cloud, edge+cloud.
- **Conway’s law**: architecture follows org chart. Inverse Conway = design org for desired architecture.
- **Include cost** trong deployment view.

Tổng kết Cụm 7

3 view chính từ SEI:

View	Câu hỏi	Audience
Module	Code organize ra sao? Depend ra sao?	Developer
C&C	Runtime ra sao? Communicate ra sao?	Architect, SRE
Allocation	Chạy ở đâu? Ai own?	DevOps, EM, Stakeholder

Plus ADR cho decision history.

Practical minimum: System Context (C4) + Container (C4) + 2-3 sequence diagrams cho critical flows + ADR cho important decisions.

Cụm tiếp: Cụm 8 - Case Studies, apply tất cả vào case thực.